


# Reporte Tercera Entrega

# MESH

**UNIVERSIDAD DEL ISTMO**

**INGENIERÍA EN SISTEMAS Y CIENCIAS DE LA COMPUTACIÓN**

**Diseño y Administración de Bases de Datos 2**

José Manuel González Corado

José David Navichoque Pineda

Juan Pablo Torres García

Guatemala, martes 20 de octubre del 2026

# INDICE

**1. INTRODUCCIÓN ..... 3**
1.1. Alcance de la entrega y repositorio ..... 3
1.2. Cómo responde este informe a la rúbrica y a los lineamientos ..... 3
1.3. Distribución de responsabilidades ..... 4
**2. BÚSQUEDA DE PRODUCTOS CON UN MOTOR DE BÚSQUEDA ..... 6**
2.1. Por qué un motor de búsqueda y su registro de decisión ..... 6
2.2. El mapeo del índice, hecho a mano ..... 6
2.3. Tolerancia a variaciones y autocompletado ..... 7
2.4. Filtros facetados con agregaciones ..... 8
2.5. Cómo se mantiene el índice al día ..... 9
2.6. Una búsqueda que no encuentra nada ..... 9
2.7. El buscador cuando Elasticsearch no está ..... 10
**3. ESTRATEGIA DE CONSISTENCIA DEL CHECKOUT ..... 11**
3.1. Los componentes que intervienen ..... 11
3.2. Los puntos de falla y su mecanismo ..... 12
3.3. Lo que la estrategia no resuelve ..... 12
**4. PRUEBA DE FALLA SIMULADA ..... 14**
4.1. Cómo se hizo ..... 14
4.2. Evidencia y comportamiento del sistema ..... 14
4.3. El hallazgo ..... 15
4.4. Muchos compradores a la vez ..... 15
**5. LA TIENDA CON UNA BASE CAÍDA ..... 17**
5.1. La idea ..... 17
5.2. De dónde sale lo que daba cada base ..... 17
5.3. Qué base cubre a cuál ..... 18
5.4. Función por función ..... 19
5.5. Las colas que guardan lo pendiente ..... 20
5.6. Si se caen dos a la vez ..... 21
5.7. El cortacircuitos ..... 22
5.8. Cómo se midió ..... 22
5.9. Lo que no tiene respaldo, a propósito ..... 23
5.10. Cómo se muestra en vivo ..... 23
**6. EVALUACIÓN DE UN MOTOR NEWSQL PARA LOS PAGOS ..... 25**
6.1. El registro de decisión ..... 25
**7. ARQUITECTURA ACTUALIZADA ..... 26**
7.1. El diagrama de arquitectura ..... 26
7.2. El diagrama entidad relación ..... 28
7.3. Cómo se levanta el sistema completo ..... 28
7.4. El dashboard del administrador ..... 28
**8. CONCLUSIONES ..... 30**
8.1. Conclusiones ..... 30

# 1. INTRODUCCIÓN

## 1.1. Alcance de la entrega y repositorio

Mesh es nuestra implementación de TiendaYa, una tienda en línea en la que varios vendedores comparten un mismo catálogo. En las dos entregas anteriores PostgreSQL quedó como fuente de verdad, MongoDB guardó el catálogo y su historial, Redis el carrito y la oferta limitada, Neo4j las relaciones entre cuentas y ClickHouse los eventos en el tiempo.

La Entrega 3 agrega dos cosas. Un motor de búsqueda que tolera errores de escritura, ordena por relevancia y calcula filtros facetados con sus propias agregaciones, y una estrategia de consistencia para el checkout, que en este punto ya toca varias bases a la vez y puede fallar a medias. Hicimos además la evaluación opcional de un motor NewSQL para los pagos.

El trabajo está en el repositorio `https://github.com/pinedahuella/Proyecto-Tienda-Base-Datos-2`, con 371 commits sobre `main` entre el 15 de agosto y el 6 de noviembre de 2026. Todo lo que se afirma en este informe sale de código que corre y de pruebas que se pueden repetir, y cada prueba indica el comando que la reproduce. La primera versión de este informe es del 25 de septiembre y la segunda del 8 de octubre.

Esta versión, del 20 de octubre, suma lo que se pidió al revisar la entrega. Que la tienda siga respondiendo cuando se apaga cualquiera de las seis bases, y que eso se pueda ver en vivo. Que los filtros del buscador se apliquen dentro de la consulta. Y que una búsqueda sin sentido, como asdfffff, no deje la página vacía. Está en las secciones 2.4, 2.6, 2.7 y 5.

El mismo día sumamos tres cosas que ayudan a mostrar lo anterior. Una demostración con cuarenta compradores a la vez sobre una venta relámpago de diez unidades, en la sección 4.4. La siembra de productos desde esta computadora hacia las bases de la otra, en la 7.3. Y un dashboard para el administrador con lo que pasa en la tienda, en la 7.4.

## 1.2. Cómo responde este informe a la rúbrica y a los lineamientos

**Tabla 1. Cada criterio de la rúbrica de la Entrega 3 con la sección que lo cubre.**

| Criterio de la rúbrica | Puntos | Sección | Evidencia medida |
|---|---|---|---|
| Buscador implementado, con tolerancia a variaciones y filtros facetados | 5 | 2 | Prueba del buscador y prueba de facetas con 26 comprobaciones en verde, filtros dentro de la consulta, buscador que sigue sin Elasticsearch y productos sugeridos cuando no hay resultados |

| Criterio de la rúbrica | Puntos | Sección | Evidencia medida |
|---|---|---|---|
| Documento de estrategia de consistencia para el checkout distribuido | 5 | 3 y 5 | `docs/CONSISTENCIA-CHECKOUT.md`, catorce puntos de falla con su mecanismo, y el mapa de qué base cubre a cuál con sus colas |
| Evidencia de la prueba de falla simulada y su manejo | 4 | 4 y 5 | Prueba de falla del checkout con 32 comprobaciones en verde, que desde el 17 de octubre compra sin Redis y sin MongoDB, y prueba de caídas con 51, con cada una de las seis bases apagada y congelada, y la demostración de concurrencia, cuarenta compradores sobre diez unidades sin vender de más, con Redis y sin Redis |
| Calidad del informe y del repositorio | 1 | 7 | Diagrama de arquitectura, diagrama entidad relación generado desde la base, arranque en un solo comando y dashboard del administrador con su prueba |
| Evaluación NewSQL para pagos, opcional | extra | 6 | Registro ADR-8 con cuatro alternativas y las condiciones en las que la respuesta cambiaría |

**Tabla 2. Los lineamientos generales del enunciado y dónde se cumplen.**

| Lineamiento del enunciado | Dónde se cumple |
|---|---|
| 5.1 Distribución de responsabilidades | Sección 1.3, respaldada por el historial de git |
| 5.2 Componentes de referencia | Elasticsearch, la opción de referencia del enunciado para el motor de búsqueda |
| 5.3 Registro de decisiones arquitectónicas | ADR-9 en 2.1 y ADR-8 en 6 |
| 5.4 Documentación para levantar el sistema | Sección 7.3, un solo comando que también carga el índice de búsqueda |

## 1.3. Distribución de responsabilidades

Mantuvimos el reparto de las entregas anteriores. José David Navichoque Pineda se encargó del backend y José Manuel González Corado del frontend, las migraciones, los ETL, los scripts de cada base, las pruebas, la documentación y el arranque. Cuando una parte necesitaba algo de la otra lo dejábamos escrito en `docs/BACKEND-PENDIENTE.md`, con la consulta ya probada. Los cambios del 19 y el 20 de octubre tocaron también el backend, y los hizo José Manuel con el visto bueno de José David, en cuya computadora viven las bases con las que se probó todo.

**Tabla 3. Quién hizo cada parte de la Entrega 3.**

| Componente | José David Navichoque, backend | José Manuel González, el resto |
|---|---|---|
| Buscador | `BuscadorService`, sus rutas y el worker que mantiene el índice al día | Mapeo del índice, analizador, ETL desde MongoDB, prueba del buscador y buscador de la cabecera |
| Corrección ortográfica | El tal vez quisiste decir, calculado en la misma consulta de búsqueda | La línea que lo muestra en la página de resultados |

| Componente | José David Navichoque, backend | José Manuel González, el resto |
|---|---|---|
| Filtros facetados | La ruta `GET /buscador/productos/facetado`, que lleva la consulta de facetas al backend | Consulta con agregaciones, su prueba y el panel de filtros de la página de resultados |
| Checkout | `CarritoService`, el candado, las reservas, la copia del catálogo en PostgreSQL para comprar sin MongoDB y el carrito en PostgreSQL de la migración 045 | Función `comprar_carrito()`, migraciones 029 y 030 en el arranque, documento de consistencia y prueba de falla simulada |
| Una base caída | Las bases de prueba en su computadora, apagadas y prendidas para medir | Respaldo de cada base en las otras, cortacircuitos, interruptor para apagarlas en vivo, prueba de caídas, buscador sin Elasticsearch y productos sugeridos |
| Concurrencia y dashboard | | Demostración de concurrencia, siembra de productos en las bases de la otra computadora y dashboard del administrador, con sus rutas, la exportación a Excel, PDF y correo, y su prueba |
| NewSQL y arquitectura | | Registro ADR-8, diagrama de arquitectura y diagrama entidad relación |

**Tabla 4. Commits por integrante en el historial de git al 6 de noviembre de 2026.**

| Integrante | Usuario en git | Commits |
|---|---|---|
| José Manuel González Corado | José Manuel González Corado | 248 |
| José David Navichoque Pineda | pinedahuella | 118 |
| Juan Pablo Torres García | zapatoazul | 5 |
| Total | | 371 |

# 2. BÚSQUEDA DE PRODUCTOS CON UN MOTOR DE BÚSQUEDA

## 2.1. Por qué un motor de búsqueda y su registro de decisión

Hasta la Entrega 2 un comprador solo encontraba un producto si escribía su nombre casi igual. MongoDB Community tiene búsqueda de texto con `$text`, pero no tolera errores de escritura. Encuentra la palabra o no la encuentra. En un catálogo escrito en español eso falla por dos lados, las tildes y los errores de tecleo.

**Tabla 5. Registro de decisión ADR-9**

| **ADR-9 Elasticsearch para la búsqueda del catálogo** | |
|---|---|
| **Contexto** | Buscar labtop no encontraba Laptop y buscar camara no encontraba Cámara. El catálogo vive en MongoDB, y hay productos que existen solo ahí porque los publica un vendedor desde el panel. |
| **Alternativas consideradas** | Seguir con `$text` de MongoDB, usar `pg_trgm` de PostgreSQL, o incorporar Elasticsearch. |
| **Decisión** | Elasticsearch 8, con un índice `mesh_productos` alimentado desde MongoDB. |
| **Justificación** | `$text` no tiene búsqueda difusa. `pg_trgm` sí la tiene, pero el catálogo no está en PostgreSQL y habría que copiarlo de vuelta, y tampoco da relevancia, autocompletado ni agregaciones sin construirlas a mano. Elasticsearch da las cuatro cosas y es la opción de referencia del enunciado. |
| **Consecuencias** | Una base más que mantener y un índice que es copia de MongoDB y puede quedar atrás unos segundos. Se acepta porque el índice no guarda nada propio y se puede reconstruir entero en cualquier momento. |

## 2.2. El mapeo del índice, hecho a mano

El enunciado pide un mapeo construido de forma deliberada y no el que Elasticsearch deduce solo. El nuestro está en `elastic/001_indice.json` y cada campo tiene un tipo elegido por la pregunta que va a recibir.

**Tabla 6. Los campos del índice mesh_productos y la razón de su tipo.**

| Campo | Tipo | Para qué |
|---|---|---|
| `nombre` | text con el analizador `espanol_llano` | La búsqueda normal, con tolerancia a errores |
| `nombre.escribiendo` | search_as_you_type | Sugerencias mientras la persona todavía escribe |
| `nombre.exacto` | keyword | Ordenar y agrupar sin analizar |
| `descripcion` y `variantes` | text con `espanol_llano` | Encontrar un producto por lo que dice de sí mismo |
| `id_categoria`, `categoria_padre`, `estado` | integer y keyword | Filtrar y agregar por valor exacto |

| Campo | Tipo | Para qué |
|---|---|---|
| `precio_desde`, `resenas_promedio`, `stock_total` | double e integer | Los rangos de las facetas |

El analizador `espanol_llano` pasa todo a minúsculas y quita las tildes, y nada más. No usamos el analizador `spanish` que trae Elasticsearch porque recorta las palabras a su raíz, y nuestros nombres son marcas y modelos inventados, como Vireo o Ámbar, que no ganan nada con eso. Tampoco quitamos palabras vacías, porque La Cumbre es un nombre y el filtro se comería La.

## 2.3. Tolerancia a variaciones y autocompletado

La búsqueda usa `multi_match` sobre nombre, descripción y variantes con `fuzziness` automática, y el nombre pesa el triple. La distancia de edición que se tolera crece con el largo de la palabra, así que labtop encuentra Laptop y una palabra de tres letras no se confunde con cualquier otra.

```
buscando "labtop"   ->  Laptop Ambar Campus 14
                        Laptop Vireo Air 12
                        (el teclado no sale)
```

Salida de la prueba del buscador, bash pruebas/busqueda.sh.

Para el autocompletado el mapeo ya trae `nombre.escribiendo`, del tipo `search_as_you_type`. Probamos la consulta con `bool_prefix` sobre ese campo y con lap sugiere solo productos cuyo nombre empieza así, mientras que la búsqueda difusa con el mismo texto trae 169 productos de cualquier tipo. La consulta está en `elastic/pruebas/facetas.py`, en la función `autocompletar()`. Desde el 20 de octubre la lista que se abre bajo el buscador de la cabecera usa las dos consultas juntas. El principio de palabra pesa más, así que con lap salen primero las laptops, y la difusa sigue encontrando labtop. Cada producto sale una sola vez, sin sus copias de lote. La prueba `pruebas/buscador_escribiendo.mjs` da 20 comprobaciones bien.

Cuando lo escrito no existe en el catálogo, la misma consulta pide una corrección con el `term` suggester sobre el nombre, solo para las palabras que no aparecen en el índice y de tres letras o más. La página de resultados la muestra como tal vez quisiste decir. Con frases de tres palabras o más la corrección sale poco útil, así que ahí no se muestra.

Después de esta entrega el buscador sumó una búsqueda híbrida, que mezcla las palabras con el significado de la frase. Se cuenta en el informe de la Entrega final, junto con los vectores que usa. Cuando Elasticsearch no está, los productos sugeridos de la sección 2.6 usan esos mismos vectores.

## 2.4. Filtros facetados con agregaciones

Los filtros de la página de resultados los calcula Elasticsearch con agregaciones, sobre los mismos resultados de la búsqueda. Hay cuatro facetas, categoría con `terms`, precio y calificación con `range`, y existencias con `filter`. Los tramos de precio salen de los percentiles del catálogo, con la mediana alrededor de Q500 y el percentil 90 alrededor de Q3600.

Los filtros elegidos van dentro de la consulta, en el `filter` de la `bool`, junto al texto buscado. Así Elasticsearch filtra primero y las agregaciones se calculan sobre lo que de verdad queda. Con Laptops elegida, la faceta de categoría muestra solo Laptops con su conteo, y la de precio reparte solo las laptops entre sus tramos. Dentro de una faceta las opciones se suman y entre facetas se cruzan.

Hasta el 20 de octubre los filtros iban en `post_filter`, que recorta la lista de resultados después de agregar. El panel seguía mostrando todas las categorías con sus conteos aunque la lista ya estuviera filtrada. En la revisión se pidió que el filtro se aplicara en la búsqueda misma, para que el panel describa lo que la persona tiene delante, y lo cambiamos en el backend y en la prueba.

**Tabla 7. Prueba de facetas, python elastic/pruebas/facetas.py, 26 OK y 0 FALLA, buscando laptop el 20 de octubre.**

| Comprobación | Resultado |
|---|---|
| Las categorías suman el total de resultados | 109 de 109 |
| Los tramos de precio suman el total | 109 |
| Los tramos de precio se tocan, sin huecos entre ellos | sí |
| Los tramos de la prueba son los mismos del backend | sí |
| Al elegir Laptops los resultados son los que prometía la faceta | 71 |
| Con Laptops elegida la faceta de categoría solo muestra Laptops | 71 |
| Con Laptops elegida los tramos de precio suman lo filtrado | 71 |
| Dos categorías se suman | 86 |
| Laptops con precio desde Q3000 da lo que prometía la faceta de precio | 68 |
| Con precio elegido los otros tramos quedan en cero | sí |
| Solo con existencias | 108, ninguno agotado |
| labtop encuentra sobre todo Laptops | sí |
| Un texto sin sentido no trae resultados ni facetas | sí |

Para comprobar que la prueba no aprueba sin mirar la rompimos a propósito y devolvimos los filtros a `post_filter`. Falló en 7 de las 24 comprobaciones que tenía entonces, justo en la que mira que el filtro vaya en la consulta y en las que esperan el panel recortado a lo elegido. También movimos el tramo de 500 a 1000 para que empezara en 600. La suma de los tramos

no lo notaba, porque ningún resultado caía en el hueco, así que agregamos dos comprobaciones, que los tramos se toquen entre sí y que sean los mismos del backend. Con ese cambio fallan las dos.

## 2.5. Cómo se mantiene el índice al día

El índice se alimenta desde MongoDB y no desde PostgreSQL, porque hay productos que solo existen en MongoDB. Si se construyera desde PostgreSQL, un producto que se vende y se puede reseñar no aparecería en el buscador, sin ningún error.

Cada vez que un producto cambia en MongoDB queda marcado con `pendiente_busqueda`, y un worker del backend lo pasa al índice cada 10 segundos. Además `etl/cargar_busqueda.sh` puede tirar el índice y reconstruirlo entero, y `arrancar.ps1` lo hace solo cuando el índice está vacío o desfasado. Como el índice no guarda nada propio, reconstruirlo siempre deja el mismo resultado.

## 2.6. Una búsqueda que no encuentra nada

Si alguien busca algo sin sentido, como asdffff, la página dice que no encontró nada y debajo ofrece productos que pueden interesar. Los elige Elasticsearch con una segunda consulta, más tolerante que la de la búsqueda. Acepta hasta dos letras distintas en palabras de cualquier largo y mira también el nombre de la categoría, de modo que lo que se parece a lo escrito, aunque sea de lejos, sale primero. Si nada se parece, el mismo motor ordena los productos activos por cuántas reseñas tienen, por su nota y por si hay existencias, así que siempre trae algo mientras haya un producto a la venta. Esa consulta tarda entre 7 y 23 milésimas.

Si Elasticsearch no responde, los sugeridos salen del backend. Primero los más cercanos por significado, comparando el vector de lo escrito con los vectores de los productos, sin el umbral de parecido de la búsqueda híbrida. Si eso no se puede, lo más vendido, y si tampoco, los productos con más reseñas. El vector de lo escrito se espera como mucho 3 segundos, porque con el modelo frío el primero tarda unos 8. En los dos casos los productos que son copias de un mismo lote se juntan en uno, para que las tarjetas sean productos distintos. La respuesta dice de dónde salieron, y la prueba del buscador revisa que con Elasticsearch arriba sean suyos.

Los sugeridos van aparte de los resultados. La búsqueda sigue contando cero resultados y así se guarda en ClickHouse, de modo que el reporte de búsquedas sin resultados del administrador sigue mostrando lo que la gente busca y la tienda no tiene. Si se contaran como resultados, esas

búsquedas desaparecerían del reporte sin avisar. Con un filtro elegido no se sugiere nada, porque ahí el vacío lo explica el filtro.

## 2.7. El buscador cuando Elasticsearch no está

Con Elasticsearch apagado el buscador contestaba un error a los 22 segundos. Ahora sale de la copia del catálogo en PostgreSQL, que ya tenía el nombre, la descripción, la categoría y las variantes de cada producto, mientras los precios, el stock y las notas ya vivían ahí. Lo que faltaba era comparar texto parecido, y lo agrega la migración 048 con tres extensiones que trae PostgreSQL, `unaccent`, `fuzzystrmatch` y `pg_trgm`.

La función `mesh_normalizar()` pasa todo a minúsculas y quita las tildes, igual que el analizador `espanol_llano` del índice. La función `mesh_coincide()` mide cuánto se parece una palabra buscada a la más parecida de un texto con la misma tolerancia que `fuzziness` automática. Palabras de una o dos letras tienen que ser iguales, de tres a cinco aceptan una letra distinta y de seis o más dos. Así labtop sigue encontrando Laptop. El nombre pesa el triple, como en el índice, y las palabras raras pesan más que las que están en todos lados.

Los filtros del panel y sus conteos se calculan con las mismas reglas, dentro de la consulta. Buscamos laptop, labtop, audifonos, camisa, laptop gamer, silla de comedor y asdffff con y sin Elasticsearch, y los totales y los conteos de todos los filtros salieron idénticos. Los primeros resultados son los mismos productos, y más abajo cambia el orden entre productos que se llaman igual. Sin Elasticsearch cada búsqueda tardó entre 50 y 760 milésimas. Lo único que falta en ese rato es el tal vez quisiste decir.

# 3. ESTRATEGIA DE CONSISTENCIA DEL CHECKOUT

## 3.1. Los componentes que intervienen

Comprar en Mesh es una transacción de PostgreSQL rodeada de lecturas y escrituras en otras bases que no pueden participar de ella. Un COMMIT de PostgreSQL no puede deshacer una escritura en Redis, y Redis no puede escribir en PostgreSQL. La postura del proyecto es que el dinero y el stock son transaccionales y no se negocian, y todo lo demás se acepta como eventual declarando cuánto tarda.

**Tabla 8. Los componentes del checkout en el orden en que aparecen.**

| Componente | Qué aporta | Dentro del COMMIT |
|---|---|---|
| Redis, candado de checkout | Evita el doble cobro | no |
| Redis, copia del carrito | Las líneas y sus cantidades, para contestar rápido | no |
| Redis, contador de venta relámpago | Decide primero quién aparta las unidades | no |
| PostgreSQL, pedido pendiente de la cuenta | El carrito de verdad desde el 17 de octubre | sí |
| PostgreSQL, `reserva_venta_relampago` | Las unidades apartadas, de donde Redis se reconstruye | sí |
| PostgreSQL, `producto_catalogo` y `variante_catalogo` | Nombre, estado y portada de cada variante, copiados de MongoDB | se leen en la misma base |
| PostgreSQL, `comprar_pedido()` | Pedido, líneas, stock, kardex y pago | sí |
| PostgreSQL, cola `evento_grafo` | Lo que tiene que viajar a Neo4j | sí |
| MongoDB, campos calculados | Precio, stock, ofertas y nota por producto, por la cola `evento_producto` | no, la cola sí |
| Neo4j y Elasticsearch | Grafo de reseñas e índice de búsqueda | no |

Pagar es `POST /carrito/comprar`. Bloquea el pedido pendiente de la cuenta y cada oferta con `FOR UPDATE`, siempre en orden de identificador, así dos compras de los mismos productos no se esperan una a la otra para siempre. Si Redis está, compara su copia del carrito con PostgreSQL. Después comprueba disponibilidad, reservas y precio, y llama a `cambiar_direccion_pedido()` y `comprar_pedido()` en la misma transacción. Si cualquier paso falla no queda ni pedido, ni pago, ni stock descontado.

Desde el 1 de octubre MongoDB ya no está en el camino de la compra, porque lo que el checkout leía de ahí se copia a PostgreSQL con las migraciones 029 y 030. Desde el 17 de octubre tampoco Redis. En la revisión se pidió que Redis nunca fuera la fuente de verdad, y la

migración 045 hizo del carrito el pedido pendiente de la cuenta en PostgreSQL, con Redis como copia rápida. Pagar solo necesita PostgreSQL. La sección 4.3 cuenta cómo se llegó a eso.

## 3.2. Los puntos de falla y su mecanismo

**Tabla 9. Los catorce puntos de falla de docs/CONSISTENCIA-CHECKOUT.md con el mecanismo adoptado.**

| Qué puede pasar | Qué hace el sistema | Mecanismo |
|---|---|---|
| Doble clic en pagar | Candado en Redis por cuenta, 30 segundos, y el `FOR UPDATE` del pedido pendiente | reintento idempotente |
| El precio cambió entre ver y pagar | Compara con la foto guardada en el carrito y pide confirmar | rechazo explícito |
| Se acabó el stock | La función lanza una excepción y no escribe nada | todo o nada |
| Falla la compra con unidades apartadas | Devuelve las reservas en ese momento, sin esperar a que venzan | compensación |
| La compra sale bien y Redis falla al confirmar | La venta queda cerrada igual | ventana aceptada |
| El proceso muere entre el COMMIT y el vaciado del carrito | El carrito que se compra es el pedido, y el COMMIT lo pasa a comprado. Lo que quede en Redis no coincide y se descarta | todo o nada |
| Dos compras a la vez sobre la misma oferta | `FOR UPDATE` con orden fijo | serialización |
| Los números del catálogo en MongoDB quedan viejos | Cola `evento_producto` llenada por triggers en la misma transacción y un worker que los recalcula | reintento idempotente |
| El grafo de Neo4j queda viejo | Cola `evento_grafo` y worker cada 10 segundos | reintento idempotente |
| El índice de búsqueda queda viejo | Marca `pendiente_busqueda` y worker cada 10 segundos | ventana aceptada |
| MongoDB cae durante el checkout | La compra sale igual con la copia del catálogo en PostgreSQL | copia dentro de la transacción |
| Redis cae durante el checkout | Se compra contra PostgreSQL y el `FOR UPDATE` del pedido hace de candado | todo o nada |
| Redis pierde el carrito o la venta relámpago | El carrito se reconstruye desde PostgreSQL al leerlo y la venta desde `reserva_venta_relampago` | reintento idempotente |
| La copia del carrito a PostgreSQL falla | El carrito queda desincronizado, la pantalla lo dice con un botón y el checkout no cobra hasta sincronizar | rechazo explícito |

Usamos los tres mecanismos que menciona el enunciado. Los reintentos idempotentes cubren el doble cobro, el grafo, los números del catálogo y un Redis que pierde lo que tenía, la compensación cubre las unidades apartadas de una compra que falla, y la ventana de consistencia eventual cubre todo lo que es copia, con su tiempo declarado.

## 3.3. Lo que la estrategia no resuelve

El precio no se congela al agregar al carrito. Se compara al pagar y, si cambió, el sistema pide confirmación en lugar de cobrar distinto sin avisar, pero la garantía fuerte no existe. Queda una

ventana pequeña en la copia del catálogo, un proceso que muere entre escribir en MongoDB y guardar la copia en PostgreSQL, que el arranque corrige al ponerla al día. Y hay dos repartidores de identificadores de producto, PostgreSQL y el panel del vendedor, que el arranque vigila y avisa cuando el margen se acaba.

# 4. PRUEBA DE FALLA SIMULADA

## 4.1. Cómo se hizo

La prueba `pruebas/falla_checkout.sh` compra de verdad por la API cinco veces seguidas y cambia solo lo que hay debajo. Primero con todo vivo, después con Redis parado, con MongoDB parado, con PostgreSQL parado y otra vez con todo vivo. La primera y la última corrida demuestran que la prueba sabe comprar. Sin ellas, un escenario que no compra no probaría nada.

Antes y después de cada escenario cuenta en PostgreSQL los pedidos, los pagos, el kardex y el stock de la oferta, y revisa el carrito en Redis y el candado de checkout.

Cada escenario lleva su regla. Con PostgreSQL parado la compra no debe salir, porque ahí viven el carrito, el dinero y el stock. Con MongoDB parado la compra sí debe salir desde el 1 de octubre, y con Redis parado desde el 17 de octubre. La prueba falla si no sale.

## 4.2. Evidencia y comportamiento del sistema

**Tabla 10. Prueba de falla del checkout, 32 OK y 0 FALLA, corrida el 18 de octubre con el carrito en PostgreSQL.**

| Escenario | Respuesta | PostgreSQL | Carrito |
|---|---|---|---|
| Todo vivo | 201, comprado | Pedido, pago, kardex y stock de 15 a 14 | Se vacía |
| Redis parado | 201, comprado | Pedido, pago, kardex y stock de 14 a 13 | Se vacía |
| MongoDB parado | 201, comprado | Pedido, pago, kardex y stock de 13 a 12 | Se vacía |
| PostgreSQL parado | 503, No se pudo conectar con PostgreSQL | Intacto | Entero |
| Todo vivo otra vez | 201, comprado | Vuelve a vender, stock de 12 a 11 | Se vacía |

Ninguna caída cobró a medias. Con PostgreSQL parado quedaron los mismos pedidos, pagos, kardex y stock que antes, el candado de Redis quedó suelto para reintentar de inmediato y el carrito quedó entero. La respuesta dice qué base falta, para quien revisa la consola, y la página le muestra a la persona un aviso común de que no se pudo completar la compra y que su carrito sigue guardado. Con Redis o MongoDB parado la compra salió y las cuatro tablas se movieron juntas.

Las corridas del 6 y el 11 de septiembre dieron 31 OK con otra respuesta en dos escenarios. Con Redis o MongoDB parado la compra contestaba 503 y no escribía nada. Que la parara Redis era de esperar, porque el carrito vivía solo ahí. Que la parara MongoDB es el hallazgo de la sección siguiente.

## 4.3. El hallazgo

La prueba encontró una dependencia que el documento no tenía escrita. Con MongoDB parado no se podía comprar, aunque MongoDB no participaba del COMMIT y solo aportaba el nombre y el estado de cada variante. No había inconsistencia, porque no se cobraba nada, pero sí una dependencia de disponibilidad.

La resolvimos el 1 de octubre. La migración 029 crea `producto_catalogo` y `variante_catalogo` en PostgreSQL con lo que el checkout lee, y el backend la escribe al editar un producto. La 030 agrega la cola `evento_producto`, que llenan triggers de oferta y reseña en la misma transacción, para que un worker recalcule después los números del catálogo en MongoDB. El arranque pone la copia al día en su paso 7e.

Después del cambio, el escenario de MongoDB en la prueba exige que la compra salga, y salió. Es el mismo instrumento que encontró el problema, midiendo que ya no está. El 17 de octubre pasó lo mismo con Redis. Con el carrito de verdad en PostgreSQL el escenario de Redis cambió de regla, ahora exige que la compra salga, y el 18 de octubre salió.

Esta prueba mira el momento de pagar. La sección 5 hace lo mismo con toda la tienda y con las seis bases.

## 4.4. Muchos compradores a la vez

La prueba de falla compra de una en una. Para ver el checkout con gente a la vez armamos `demo-concurrencia.ps1`, que trabaja por la API con cuentas de verdad. Cuarenta compradores navegan juntos por el catálogo, la búsqueda y la ficha. Después los cuarenta piden en el mismo instante una unidad de una venta relámpago que tiene diez, y los que ganaron pagan todos a la vez. Quién gana lo decide el script atómico de Redis, y si Redis no está, PostgreSQL con `FOR UPDATE`. La aplicación no lleva candados propios.

**Tabla 11. Demostración de concurrencia del 20 de octubre, con el backend en esta computadora y las bases en la de José David.**

| Paso | Con todo arriba | Con Redis apagado |
|---|---|---|
| Navegar, 120 páginas | 120 bien, mediana 2.1 s | 120 bien, mediana 1.8 s |

| Paso | Con todo arriba | Con Redis apagado |
|---|---|---|
| Cuarenta piden una de diez | 10 la consiguen y 30 reciben que ya no hay, mediana 4.2 s | 10 la consiguen y 30 reciben que ya no hay, mediana 3.6 s |
| Los diez pagan a la vez | 10 compras con 201, mediana 4.3 s | 10 compras con 201, mediana 1.8 s |
| Stock en PostgreSQL | De 10 a 0, con 10 pedidos pagados y 10 movimientos de stock | De 10 a 0, con 10 pedidos pagados y 10 movimientos de stock |

En las dos corridas se vendieron exactamente las diez unidades y el stock quedó en cero, nunca negativo. Los tiempos incluyen el viaje por la red hasta la otra computadora, con las cuarenta peticiones llegando juntas. Al terminar, la demostración cancela sus pedidos, devuelve el stock y borra de ClickHouse lo que movió, para que no cambien lo más vendido ni las tendencias. Usa un vendedor propio que el ETL de compras no copia, así el siguiente arranque no deshace esa limpieza.

# 5. LA TIENDA CON UNA BASE CAÍDA

## 5.1. La idea

La prueba de la sección 4 mira una sola cosa, que ninguna caída cobre a medias. En la revisión se pidió más. Que con cualquiera de las seis bases apagada la tienda siga respondiendo con las que quedan, y que se pueda mostrar en vivo con la tienda abierta.

Lo que lo hace posible es que PostgreSQL es la verdad. El dinero, el stock y los pedidos solo se escriben ahí, en una transacción. Las otras bases guardan copias o datos de apoyo, y cada copia tiene de dónde rehacerse. Cuando una se cae, el backend pregunta a la fuente de esa copia en el momento, y la tienda sigue mostrando lo mismo, a veces con un poco menos de detalle.

Ninguna pantalla nombra la base que se cayó, porque a quien compra no le sirve saberlo. Cuando algo de verdad no se puede, como pagar sin PostgreSQL, ve un aviso común que dice que no se pudo completar y que intente en un momento. El nombre de la base queda en el registro del backend, para quien tenga que arreglarlo.

## 5.2. De dónde sale lo que daba cada base

**Tabla 12. Lo que pasa con cada base apagada, del documento docs/QUE-PASA-SI-SE-CAE-UNA-BASE.md.**

| Base caída | Lo que daba | De dónde sale mientras no está | Lo que se pierde |
|---|---|---|---|
| PostgreSQL | Pedidos, pagos, stock, cuentas y carrito | La sesión abierta sigue por su token firmado, el perfil y el carrito salen de la copia en Redis, y el catálogo, la ficha, el buscador y las recomendaciones de las otras bases | Nada. No se puede pagar, agregar al carrito ni entrar con otra cuenta hasta que vuelva |
| MongoDB | La ficha completa de cada producto | La copia del catálogo en PostgreSQL, sin las características | El registro de las entradas a la tienda de ese rato |
| Redis | Rankings calculados, ofertas destacadas y copia del carrito | Los cuatro rankings se calculan en el momento en ClickHouse con la misma consulta del worker, las ofertas destacadas salen de PostgreSQL y el carrito del de PostgreSQL | Los contadores que vigilan entradas sospechosas |
| Elasticsearch | El buscador y sus filtros | La copia del catálogo en PostgreSQL con la regla de texto de la sección 2.7 | Nada, los cambios esperan en una cola |
| Neo4j | Para ti y quienes lo compraron también llevaron | Las mismas dos consultas sobre los pedidos en PostgreSQL | Nada, los cambios esperan en una cola |
| ClickHouse | Vistas, búsquedas y tendencias | Lo más vendido y lo más visto siguen en Redis. Viste hace poco y las tendencias del administrador no tienen otra fuente y avisan | Las vistas, búsquedas y agregados al carrito de ese rato |

Con PostgreSQL caída quien ya había entrado sigue dentro. Su sesión es un token firmado que dura hasta dos horas, y cuando PostgreSQL no contesta el backend se lo cree en vez de preguntar si la cuenta sigue activa. Si PostgreSQL contesta que la cuenta está suspendida se corta como siempre. Solo se le cree cuando no se pudo preguntar. La ficha de un producto se ve, y en lugar de los vendedores dice que ahora no se puede mostrar quién lo vende ni comprarlo.

Con Redis caída, comparar las ocho vitrinas de la portada con y sin Redis encontró algo que estaba mal con Redis arriba. La copia de las ofertas destacadas no se actualizaba cuando una compra bajaba el stock, y la vitrina podía mostrar unidades que ya no había. Ahora se refresca con cada cambio de una oferta.

## 5.3. Qué base cubre a cuál

La tabla anterior se lee base por base. La figura siguiente lo junta en un solo dibujo. Cada flecha va de la base que cubre a la base caída, y en la caja punteada están las cuatro cosas que no tienen respaldo, que se explican en 5.9.

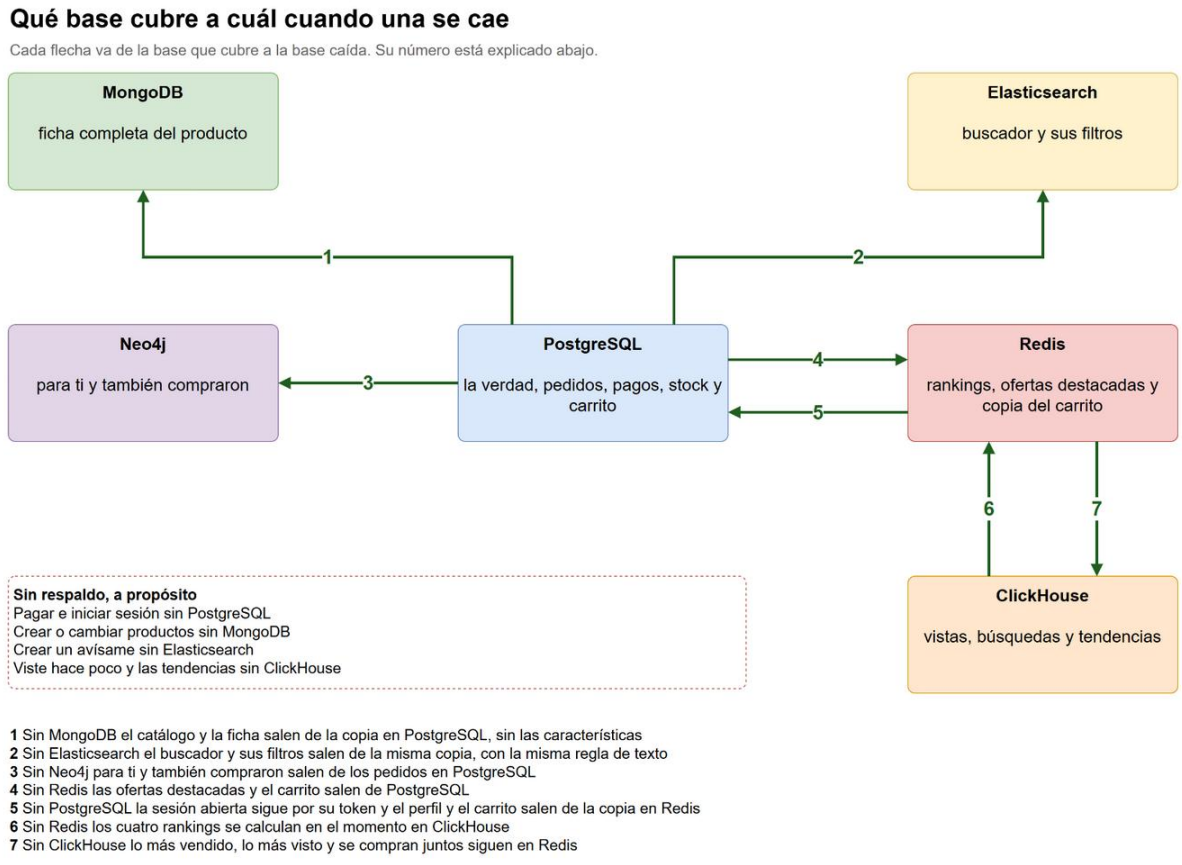



Figura 1. Qué base cubre a cuál cuando una se cae, generado con docs/generar-respaldos.py.

PostgreSQL es la que más cubre, cuatro de las seis flechas salen de ella. Es lógico, porque las otras bases guardan copias de datos que nacen ahí. La única que cubre a PostgreSQL es Redis, y solo en lo que hace falta para que quien ya entró siga mirando su carrito. Redis y ClickHouse se cubren entre sí, porque los rankings se calculan en ClickHouse y se guardan en Redis.

**Tabla 13. Qué base cubre a cada una, con qué datos y cómo se ve en el registro del backend.**

| Base caída | La cubre | Con qué | Cómo se nota en el registro |
|---|---|---|---|
| PostgreSQL | Redis, y el token firmado de la sesión | La copia del perfil y del carrito que se guarda al entrar | Una línea de `SesionGuard` como mucho por minuto |
| MongoDB | PostgreSQL | `producto_catalogo` y `variante_catalogo`, que el backend escribe en la misma operación que MongoDB | `CatalogoRespaldo` al entrar y al salir |
| Redis | PostgreSQL y ClickHouse | La tabla de ofertas, el pedido pendiente de cada cuenta y la misma consulta del worker de rankings | `Cortacircuitos` dice que Redis no responde y que volvió |
| Elasticsearch | PostgreSQL | La copia del catálogo con `mesh_normalizar()` y `mesh_coincide()` de la migración 048 | `BuscadorRespaldo` al entrar y al salir |
| Neo4j | PostgreSQL | Las líneas de los pedidos comprados, enviados o entregados, que son lo mismo que los nodos de compra del grafo | `RecomendacionesRespaldo` al entrar y al salir |
| ClickHouse | Redis, en parte | Los rankings que ya estaban guardados | `Cortacircuitos` y las líneas de `EventosService` que no pudieron registrar la vista |

## 5.4. Función por función

Lo mismo visto desde la tienda. Cada fila es algo que la persona usa, la base que lo da con todo arriba, lo que pasa si esa base cae y lo que pasa si también cae la que la cubre.

**Tabla 14. Cada función de la tienda con la base que la da y lo que pasa con una o dos bases caídas.**

| Función | La da | Si esa base cae | Si cae también la que cubre |
|---|---|---|---|
| Pagar | PostgreSQL | No se cobra. Aviso común y el carrito queda guardado | No tiene respaldo |
| Iniciar sesión | PostgreSQL | Aviso, y se puede mirar la tienda sin cuenta | No tiene respaldo |
| Ver el carrito | PostgreSQL, con copia en Redis | Sin Redis sale de PostgreSQL. Sin PostgreSQL sale de Redis y no se puede cambiar | Aviso |
| Catálogo, categorías y ficha | MongoDB | Sale de la copia en PostgreSQL, sin la pestaña de detalles | Aviso |
| Vendedores y precios en la ficha | PostgreSQL | La ficha dice que ahora no se puede mostrar quién lo vende ni comprarlo | No tiene respaldo |

| Función | La da | Si esa base cae | Si cae también la que cubre |
|---|---|---|---|
| Buscador y filtros | Elasticsearch | Sale de la copia en PostgreSQL, sin el tal vez quisiste decir | Aviso |
| Sugeridos de una búsqueda vacía | Elasticsearch | Los más cercanos por vectores en PostgreSQL, después lo más vendido y después los de más reseñas | La página dice que no encontró nada, sin sugeridos |
| Para ti y también compraron | Neo4j | Salen de los pedidos en PostgreSQL, iguales | Aviso |
| Parecidos a este | PostgreSQL con pgvector, y la tarjeta de MongoDB | Sin MongoDB la tarjeta sale de la copia. Sin PostgreSQL no salen | Aviso |
| Lo más vendido, lo más visto y se compran juntos | Redis | Se calculan en el momento en ClickHouse | Aviso |
| Ofertas destacadas y venta relámpago de la portada | Redis | Salen de la tabla de ofertas en PostgreSQL | Aviso |
| Viste hace poco y tendencias del administrador | ClickHouse | Avisan | No tiene respaldo |
| Avísame cuando llegue | Elasticsearch | No se puede crear uno nuevo | No tiene respaldo |
| Crear o cambiar un producto y ver su historial | MongoDB | No se puede hasta que vuelva | No tiene respaldo |
| Ofertas en el panel del vendedor y del administrador | PostgreSQL | Sin MongoDB se ven y se cambian igual, con el nombre de la copia | Aviso |
| Registro de cada entrada a la tienda | MongoDB | Se entra igual y la entrada de ese rato no queda anotada | No tiene respaldo |
| Contadores de entradas sospechosas | Redis | Se entra igual y esos intentos no cuentan para el fraude | No tiene respaldo |

## 5.5. Las colas que guardan lo pendiente

Lo que tiene que llegar a otra base y no puede perderse se anota primero en una cola, dentro de la misma transacción que el cambio. Casi todas viven en PostgreSQL, y la del buscador de productos es una marca en el mismo documento de MongoDB. Un worker del backend lo lleva a la otra base y marca la fila como procesada. Si la otra base está caída la fila se queda esperando, y cuando vuelve la cola baja sola. Por eso en la tabla de 5.2 Elasticsearch y Neo4j no pierden nada.

**Tabla 15. Las colas que mantienen al día las otras bases.**

| Cola | Qué lleva | A qué base | Cada cuánto |
|---|---|---|---|
| `evento_producto` | Precio, stock, ofertas y nota de cada producto | MongoDB | 5 s |
| `evento_grafo` | Las reseñas | Neo4j | 10 s |
| `evento_grafo_compra` | Las compras | Neo4j | 10 s |

| Cola | Qué lleva | A qué base | Cada cuánto |
|---|---|---|---|
| `evento_oferta` | Las ofertas, y después se comparan con los avísame | Elasticsearch | 5 s |
| Marca `pendiente_busqueda` en MongoDB | Los productos que cambiaron | Elasticsearch | 10 s |
| `etl/cargar_compras.sh` | La copia de las compras y devoluciones | ClickHouse | En cada arranque |

Para ver una cola esperando basta contar las filas sin procesar. Con Neo4j apagada el 19 de octubre se hizo una compra, quedó en la cola y al volver bajó a cero en 15 segundos. Con Elasticsearch apagada la cola de ofertas bajó a cero en un minuto.

```
SELECT count(*) FROM evento_grafo_compra WHERE procesado_en IS NULL;
SELECT count(*) FROM evento_oferta WHERE procesado_en IS NULL;
SELECT count(*) FROM evento_producto WHERE procesado_en IS NULL;
```

Las consultas para ver cuántos cambios esperan a cada base.

Las vistas, las búsquedas y los agregados al carrito que van a ClickHouse no pasan por cola, a propósito. Se mandan sin esperar respuesta para que quien compra no espere por un dato que solo sirve para analizar. Si ClickHouse no está, los de ese rato se pierden.

## 5.6. Si se caen dos a la vez

Cada copia tiene un solo respaldo. Si caen dos bases que no se cubren entre sí, cada una sigue con el suyo. Lo comprobamos con la demostración de concurrencia con Elasticsearch, MongoDB y Redis apagadas al mismo tiempo. Cuatro compradores pidieron dos unidades de una venta relámpago, ganaron dos, pagaron en 0.7 segundos y el stock no bajó de cero.

Si cae una base junto con la que la cubre, esa función avisa y la tienda sigue con lo demás. Estos casos no los medimos uno por uno, salen de cómo está hecho cada respaldo en el backend.

- MongoDB y PostgreSQL. El catálogo y la ficha no tienen de dónde salir y avisan.
- Elasticsearch y PostgreSQL. El buscador avisa.
- Neo4j y PostgreSQL. Para ti y también compraron avisan.
- Redis y ClickHouse. Los rankings de la portada avisan, porque Redis guarda la copia y ClickHouse es de donde sale.
- Con PostgreSQL en cualquiera de las parejas no se puede pagar, igual que sola.

## 5.7. El cortacircuitos

Una base caída no siempre dice que no está. Si la red se corta a mitad de camino, la conexión se queda esperando, y antes de este cambio cada petición esperaba entre 10 y 30 segundos su límite antes de rendirse. Ahora cada base tiene un cortacircuitos. La primera petición después de una caída espera lo que tarda la base en no contestar, y durante los 15 segundos siguientes el backend ni le pregunta y va directo al respaldo. Pasado ese rato la vuelve a probar, y si contesta la usa otra vez sola, sin reiniciar nada.

Con cualquiera de las seis bases congelada esa primera espera es de unos 3 segundos. MongoDB y ClickHouse tardaban más. La conexión ya abierta a MongoDB esperaba a que el driver notara la caída, entre 9 y 15 segundos, y la consulta de viste hace poco esperaba los 8 segundos del cliente de ClickHouse. Ahora esas dos consultas tienen su propio límite de 3 segundos, igual que la búsqueda en Elasticsearch, y con cualquiera de las dos congelada la primera petición tarda entre 3.0 y 3.7 segundos. En todos los casos la página sigue y las peticiones siguientes ya no esperan.

Cada cortacircuitos deja una línea en el registro del backend cuando la base no responde y otra cuando vuelve, y la ruta `GET /db/cortacircuitos` dice qué base está en pausa y desde cuándo. Solo contesta a la cuenta del administrador, la página no la usa.

La batería de pruebas encontró un error en el de PostgreSQL. Con 60 cuentas entrando a la vez, las peticiones se quedaban esperando una conexión libre, el cortacircuitos lo tomaba como base caída y la tienda pasaba 15 segundos sin su base principal estando sana. Ahora solo cuenta como caída un fallo de conexión real. Una fila de espera llena o una consulta larga es carga, y se atiende como carga.

## 5.8. Cómo se midió

Para apagar una base sin tocar la computadora donde vive pusimos un interruptor entre el backend y cada base, `remoto/interruptor.cjs`. Cada base se puede dejar apagada, que rechaza la conexión al instante como un `docker stop`, o congelada, que acepta la conexión y no contesta, como una red cortada. La segunda es la difícil, porque el backend tiene que darse cuenta solo.

**Tabla 16. Las comparaciones son del 19 y el 20 de octubre y los tiempos del 20 de octubre en la madrugada, con las bases en la computadora de José David.**

| Base caída | Comparado con todo arriba | Apagada | Congelada, primera petición | Vuelve sola en |
|---|---|---|---|---|
| PostgreSQL | Sesión, carrito, ficha, catálogo y recomendaciones contestan, pagar y entrar avisan | menos de 0.1 s, pagar avisa en 0.2 s | 3.0 s | 15.5 s |
| MongoDB | 362 de 363 comparaciones iguales en 71 pedidos, la distinta avisa a propósito | la primera 3.2 s, después menos de 0.4 s | 3.2 a 3.7 s | 14 s |
| Redis | Las ocho vitrinas iguales, mismos productos y mismo orden | menos de 0.11 s | 3.2 s | 14 s |
| Elasticsearch | Totales y conteos de filtros idénticos en siete búsquedas | 0.2 a 0.3 s | 3.3 s | 15 s |
| Neo4j | Ocho de ocho recomendaciones iguales, con el mismo porqué | menos de 0.4 s | 3.2 s | 15 s |
| ClickHouse | Lo más vendido sigue, viste hace poco y tendencias avisan | avisa en menos de 0.04 s | 3.0 a 3.5 s | 15 s |

La prueba `pruebas/caidas.mjs` hace todo esto sola. Apaga cada base, una por una, comprueba lo que dice la tabla anterior y lo que tiene que avisar, la vuelve a prender y comprueba que la tienda la usa otra vez sin reiniciar nada. Dio 51 comprobaciones bien y ninguna mal con las bases apagadas, y otras 51 con las bases congeladas.

## 5.9. Lo que no tiene respaldo, a propósito

- Iniciar sesión sin PostgreSQL. Copiar las cuentas a otra base pondría las contraseñas en dos lugares que proteger, y una cuenta suspendida podría entrar con la copia vieja. Quien no había entrado puede mirar la tienda sin cuenta.
- Viste hace poco y las tendencias del administrador sin ClickHouse. Las vistas solo viven ahí. Se mandan sin esperar respuesta y sin cola, para que quien compra no espere por algo que solo sirve para analizar, y las de ese rato se pierden.
- Crear un avísame sin Elasticsearch, porque la comparación de cada oferta contra los avisos guardados es del propio motor. Tampoco se busca entre las ofertas del panel del vendedor, que sigue viendo y cambiando sus ofertas en la lista de siempre.
- Crear o cambiar productos sin MongoDB, porque la ficha completa vive ahí.

## 5.10. Cómo se muestra en vivo

El panel de comandos del proyecto tiene un grupo, Caída de una base, con un botón para preparar la demostración, que reinicia el backend a través del interruptor, y botones para

apagar, congelar y prender cada base. Lo mismo se hace desde una terminal con `remoto/caidas.ps1`. Al final de `docs/QUE-PASA-SI-SE-CAE-UNA-BASE.md` hay un guion de unos diez minutos que recorre las seis bases con la tienda abierta, empezando por Redis y terminando con PostgreSQL, que es la que manda.

# 6. EVALUACIÓN DE UN MOTOR NEWSQL PARA LOS PAGOS

## 6.1. El registro de decisión

El pago en Mesh es una fila que nace dentro de la misma transacción que el pedido, el descuento de stock y el kardex. Evaluamos si convenía llevarlo a un motor NewSQL, que reparte transacciones ACID entre varios nodos con consenso Raft. El análisis completo está en `docs/DECISION-NEWSQL-PAGOS.md`.

**Tabla 17. Registro de decisión ADR-8**

| **ADR-8 Pagos en un motor NewSQL** | |
|---|---|
| **Contexto** | Un solo PostgreSQL con pedidos y pagos en decenas o pocos cientos. El único punto de contención real es la venta relámpago, que ya se resolvió en la Entrega 2 con un contador atómico en Redis delante de PostgreSQL. |
| **Alternativas consideradas** | Quedarse en PostgreSQL, mover solo los pagos a NewSQL, mover todo el núcleo transaccional a CockroachDB, o moverlo a YugabyteDB. |
| **Decisión** | Se queda PostgreSQL. |
| **Justificación** | Mover solo los pagos separa el cobro del stock y obliga a una saga entre motores, justo lo que el checkout evita hoy. Moverlo todo conserva la atomicidad pero reparte entre nodos una carga que un solo nodo aguanta de sobra, vuelve más lenta la fila caliente de la venta relámpago porque cada escritura pasa por consenso, pide al menos tres nodos para tolerar fallos y obliga a portar funciones PL/pgSQL y triggers que en CockroachDB tienen soporte parcial. |
| **Consecuencias** | El checkout sigue siendo una transacción local y fácil de probar. Se asume que PostgreSQL es un punto único de falla, y si cae no se puede comprar hasta que vuelva, sin cobrar a medias. La sección 5 cuenta lo que sí sigue funcionando en ese rato. |

La respuesta cambiaría con compradores en varias regiones que necesiten escribir cerca, con un volumen de pedidos que un nodo ya no aguante medido y no supuesto, o con un requisito de disponibilidad que no admita perder la compra durante la caída de un servidor. Para el tercer caso hay un paso más barato antes de NewSQL, una réplica de PostgreSQL con conmutación automática. Si llegara el caso, YugabyteDB sería la primera opción porque corre las funciones y triggers que hoy sostienen el checkout.

# 7. ARQUITECTURA ACTUALIZADA

## 7.1. El diagrama de arquitectura

Las flechas continuas son lo que hace el backend con cada base, y las punteadas y numeradas son datos que viajan de una base a otra. En esta entrega entra el índice de búsqueda con su worker. El diagrama está al 20 de octubre, así que también trae la copia del catálogo en PostgreSQL con su cola, lo que sumó la Entrega final y los cambios de octubre. Los respaldos de la sección 5 no agregan flechas. Usan conexiones del backend que ya están dibujadas, solo que las consultan cuando falta la base de siempre.

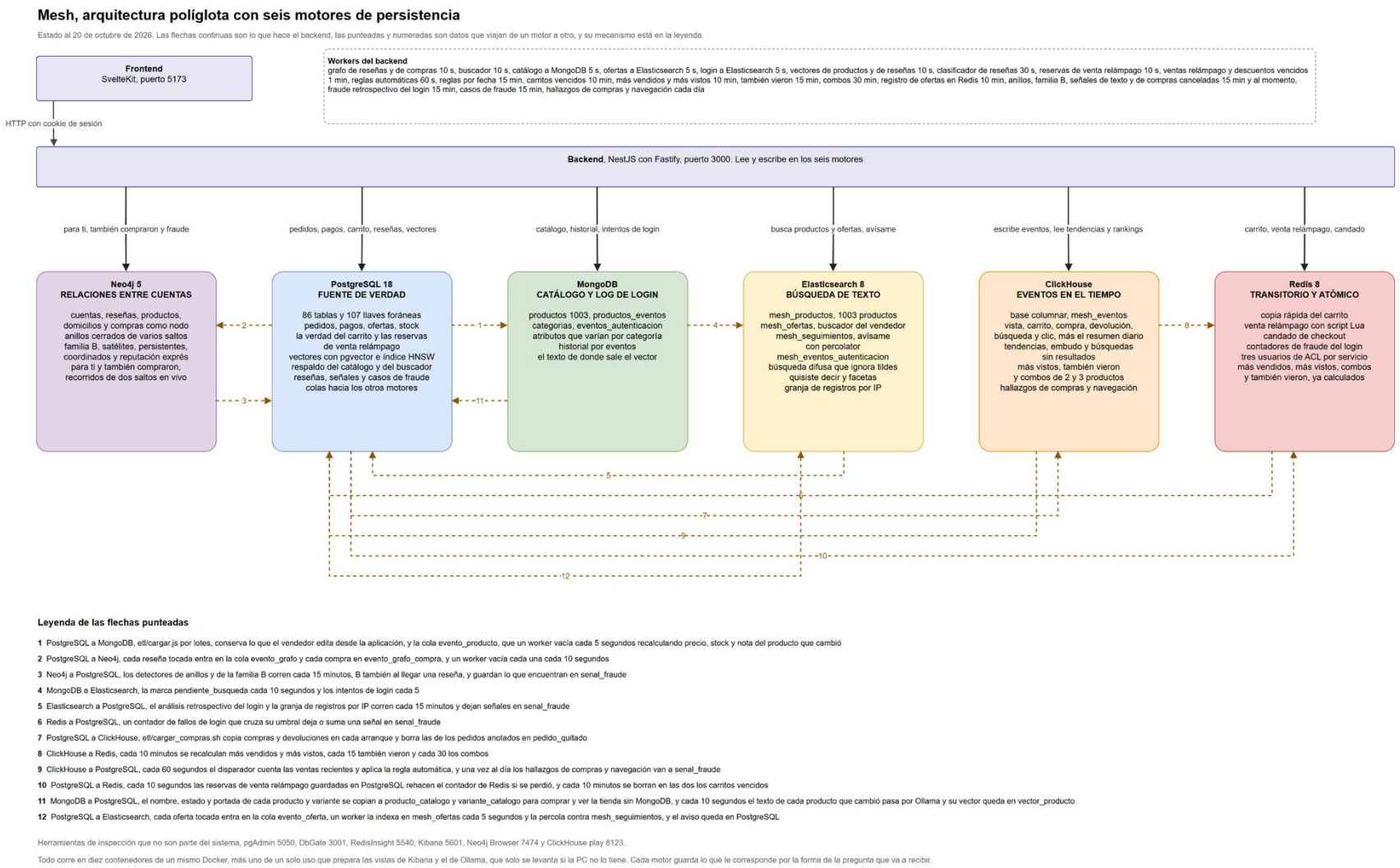



Figura 2. Arquitectura de Mesh al 20 de octubre de 2026, generada con docs/generar-arquitectura.py.

## 7.2. El diagrama entidad relación

PostgreSQL tiene hoy 86 tablas y 107 llaves foráneas, de las migraciones 001 a 048, contadas el 20 de octubre en la base donde corre la tienda. El diagrama se genera leyendo el esquema de la base en funcionamiento con `docs/generar-er.py`, y el script falla si una tabla o una llave foránea queda sin dibujar, así que no puede quedar atrás de las migraciones.

## 7.3. Cómo se levanta el sistema completo

Todo se levanta con `.\arrancar.ps1`. El script prende los contenedores, corre las migraciones, carga los datos, reconstruye el catálogo en MongoDB, el índice de búsqueda y el grafo, y al final cuenta lo que quedó en cada base contra PostgreSQL en lugar de suponer que salió bien. El backend y el frontend se arrancan después con `npm run start:dev` y `npm run dev`. La guía completa está en `docs/instalacion.md`.

Las bases también pueden vivir en otra computadora. Con `usar-bases-remotas.ps1` el backend apunta a ellas, y con `source remoto/activar.sh` las pruebas corren contra esa computadora como si las bases estuvieran aquí. Así se midió todo lo de la sección 5.

Desde el 20 de octubre `arrancar.ps1` también corre entero contra las bases de la otra computadora. Así se siembran y se borran productos desde aquí, en PostgreSQL, MongoDB y el buscador a la vez, y el panel de comandos del proyecto lo hace con un botón. Lo que borra volúmenes de Docker, como rehacer MongoDB o borrar todo, se niega en ese modo y se hace en la computadora que tiene las bases.

## 7.4. El dashboard del administrador

El panel del administrador abre en un dashboard con lo que pasa en la tienda en el periodo que se elija, comparado con el periodo anterior del mismo largo. Tiene seis vistas que se ven de una en una, ventas, clientes, vendedores y productos, búsquedas, cancelaciones y pedidos.




Figura 3. La vista de ventas del dashboard en lo que va del año, el 20 de octubre de 2026.

Cada cifra sale de la base que la tiene. Lo cobrado, los pedidos, las categorías, los departamentos, las cancelaciones y el mapa de calor por día y hora salen de PostgreSQL, con la misma regla que la pestaña de reportes. Una venta es un pedido comprado, enviado o entregado, y lo cobrado es un pago pagado. El embudo de compra y las búsquedas salen de ClickHouse, y el embudo tiene vistas registradas desde el 7 de octubre de 2026. Los días y las horas van en hora de Guatemala.

Lo que se ve se puede bajar en Excel o en PDF, o mandar por correo a una o varias direcciones, con las secciones que se elijan. La prueba `pruebas/dashboard.mjs` da 35 comprobaciones bien y ninguna mal. Revisa que todas las vistas sumen lo mismo, que cuadren con la pestaña de reportes y con PostgreSQL contado aparte, que el Excel y el PDF traigan esos números y que el correo rechace lo inválido sin mandar nada.

# 8. CONCLUSIONES

## 8.1. Conclusiones

El motor de búsqueda resolvió algo que ninguna de las bases anteriores hacía bien, encontrar un producto aunque se escriba mal su nombre, y además calcula los filtros de la página de resultados con sus propias agregaciones. El mapeo se hizo campo por campo y el analizador se eligió mirando nuestros nombres de producto.

En el checkout aprendimos que la consistencia se decide punto por punto. El dinero y el stock quedaron en una sola transacción y todo lo demás tiene un mecanismo declarado. La prueba de falla confirmó que ninguna caída cobra a medias, encontró una dependencia con MongoDB que no teníamos escrita, y después de quitarla midió que hoy se compra solo con PostgreSQL. Con cuarenta compradores a la vez sobre diez unidades se vendieron exactamente diez, con Redis y sin él.

Lo que más nos enseñó la revisión fue apagar las bases de verdad. Que cada copia tuviera de dónde rehacerse ya estaba en el diseño, pero hasta probarlo la tienda se quedaba esperando a una base caída hasta 30 segundos y varias pantallas quedaban vacías. Con un respaldo por base y un cortacircuitos, hoy la tienda sigue con cualquiera de las seis apagada o congelada, solo la primera petición espera y la base vuelve a usarse sola a los 15 segundos de prenderla. Además, comparar cada pantalla con y sin su base encontró un error que estaba ahí con todo arriba, la copia de las ofertas destacadas que no bajaba con las compras.

La evaluación de NewSQL terminó en no migrar, y lo sostenemos con los mismos criterios que las decisiones anteriores. A este volumen un motor distribuido resolvería un problema que no tenemos y haría más lento el que sí tenemos.